

Arbeiten mit Bildern

Import der Bibliotheken

```
In [1]: import numpy as np
import scipy
import cv2
import matplotlib.pyplot as plt
import plotly
import plotly.express as px
import plotly.graph_objects as go
from plotly.subplots import make_subplots
```

Laden eines Testbildes

```
In [2]: image_baboon=cv2.imread('./Images/baboon.png', cv2.IMREAD_COLOR_RGB)
# image taken from https://github.com/scijs/baboon-image/blob/master/baboon.png
# image is loaded as BRG image. convert to RGB
```

Anzeigen der Datenstruktur des Bildes

Dieses Farb - Bild wird als numpy ndarray, einer $v \times u \times 3$ Zahlen - Matrix geladen.

Die Koordinaten u, v repraesentieren hier die Pixelposition im Bild, die 3. Koordinate die jeweiligen Farbkanäle Rot, Gruen, und Blau (rgb)

Bildgröße und Datentyp können durch folgende Befehle abgefragt werden.

```
In [3]: dataType = image_baboon.dtype
imageShape = image_baboon.shape
print(f'Datentyp des Bildes: {dataType}')
print(f'format des Bildes: {imageShape}')
```

```
Datentyp des Bildes: uint8
format des Bildes: (512, 512, 3)
```

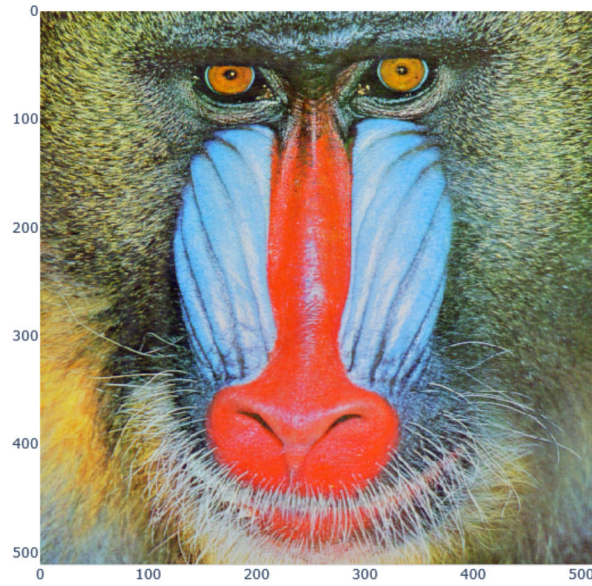
Anzeigen des Bildes

Inline Anzeige des Bildes

Bewegen den Mauszeiger über das Bild, dann können Sie sowohl die Werte der Farbkanäle als auch die dazugehörigen Pixelkoordinaten auslesen.

```
In [4]: # Anzeige des Bildes
height, width = image_baboon.shape[:2]
fig = px.imshow(image_baboon)
fig.update_layout(width=width, height=height + 50, margin=dict(l=0, r=0, t=50, b=0))
```

```
fig.show()
```



Farbkanäle einzeln Anzeigen

Die Farbkanäle des Bildes lassen sich einzeln ansteuern

Roter Kanal `image_baboon[:, :, 0]`

Grüner Kanal `image_baboon[:, :, 1]`

Blauer Kanal `image_baboon[:, :, 2]`

```
In [5]: image_baboon_red = image_baboon[:, :, 0]
image_baboon_green = image_baboon[:, :, 1]
image_baboon_blue = image_baboon[:, :, 2]

## Plot the images
height, width = image_baboon.shape[:2]

# Anzeige des roten Kanals
fig = px.imshow(image_baboon_red, color_continuous_scale='gray', title='Roter Farbk
fig.update_layout(width=width, height=height + 50, margin=dict(l=0, r=0, t=50, b=0)
fig.show()

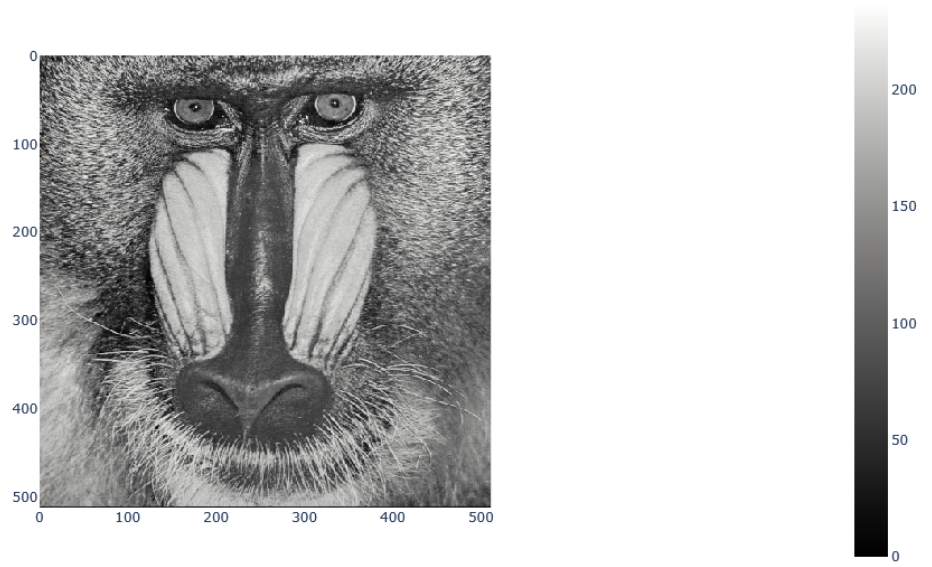
# Anzeige des grünen Kanals
fig = px.imshow(image_baboon_green, color_continuous_scale='gray', title='Gruener F
fig.update_layout(width=width, height=height + 50, margin=dict(l=0, r=0, t=50, b=0)
fig.show()

# Anzeige des blauen Kanals
fig = px.imshow(image_baboon_blue, color_continuous_scale='gray', title='Blauer Far
fig.update_layout(width=width, height=height + 50, margin=dict(l=0, r=0, t=50, b=0)
fig.show()
```

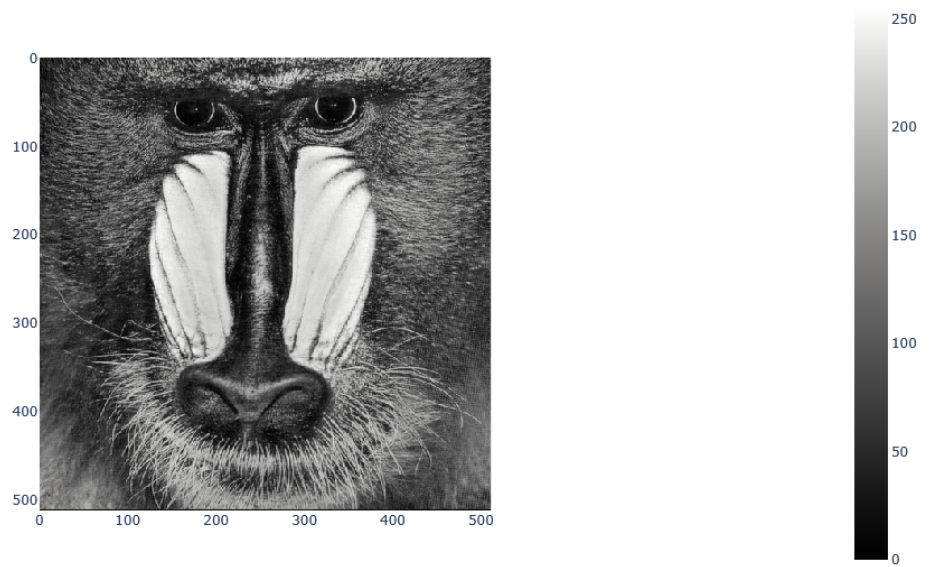
Roter Farbkanal



Gruener Farbkanal



Blauer Farbkanal



Filterung nach einer Speziellen Farbe / Farbbereich

mit der Funktion `cv2.inRange` lässt sich das Bild nach einem bestimmten Farbbereich filtern. Die Funktion `cv2.inRange(image, lowerb, upperb)` Dabei werden die Pixel herausgefiltert deren Farbwert (r, g, b) zwischen der unteren Farbgrenze (lowerb) und der oberen Farbgrenze (upperb) liegt. Im Nachfolgenden Beispiel ist: `lowerb = (200, 0, 0)` und `upperb = (255, 100, 100)` Somit filtert diese Funktion Farbwerte: $200 < r \leq 255$ und $0 < g < 100$ und $0 < b < 100$

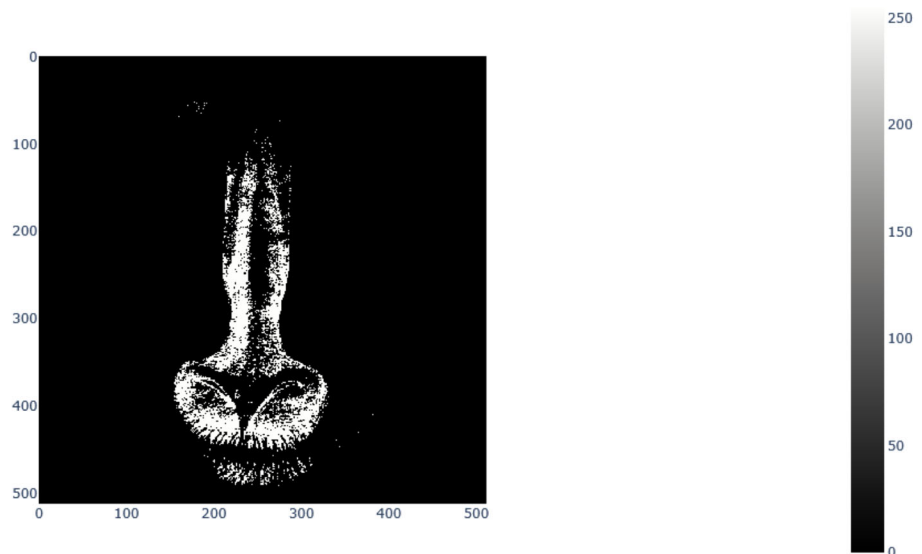
Nase

Die Rote Nase lässt sich leicht herausfiltern, da diese kaum andere Farbkomponenten besitzt

```
In [6]: nose = cv2.inRange(image_baboon, (150, 0, 0), (255, 80, 80))

# Anzeige der Nase
height, width = nose.shape[:2]
fig = px.imshow(nose, color_continuous_scale='gray', title='Maskierte Nase')
fig.update_layout(width=width, height=height + 50, margin=dict(l=0, r=0, t=50, b=0))
fig.show()
```

Maskierte Nase



Aufgabe

Augen

Nachdem eben die Nase des Affen gefiltert wurde sollen nun die Augen gefiltert werden. Versuchen Sie den Filter so anzupassen dass nur die zu den orangenen Augen gehörenden Pixel detektiert werden. Nutzen Sie hierfür das oben stehende Farbbild und lesen Sie die einzelnen Farbwerte mit dem Mauszeiger ab.

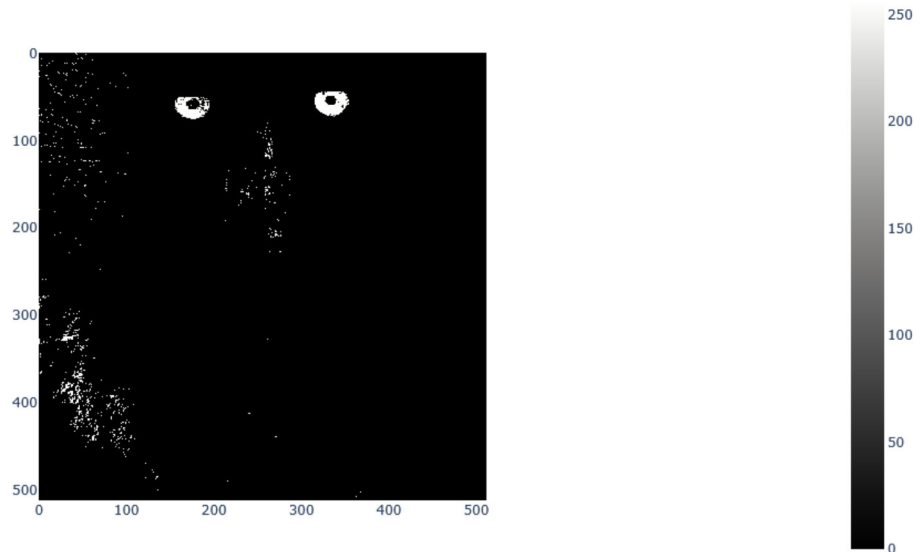
Achtung Sie werden feststellen dass dies etwas schwerer umsetzbar ist, die orangenen Augen

lassen sich nicht so einfach von den restlichen Farbtönen trennen

```
In [7]: ## Verändern Sie die Filterwerte durch Anpassen der unteren Schranke und der oberen
jaw = cv2.inRange(image_baboon, (150, 100, 0), (255, 255, 50))

# Anzeige der Nase
height, width = jaw.shape[:2]
fig = px.imshow(jaw, color_continuous_scale='gray', title='Maskiertes Bild')
fig.update_layout(width=width, height=height + 50, margin=dict(l=0, r=0, t=50, b=0))
fig.show()
```

Maskiertes Bild



Wechsel des Farbraums

Der RGB-Farbraum entspricht nicht unbedingt der menschlichen Farbwahrnehmung, in dem Sinne, dass ähnliche Farben (r,g,b) auch zahlenmäßig nahe beieinanderliegen. Der HSV Farbraum parametrisiert die Farben so, dass ähnliche Farben jetzt auch örtlich nebeneinander liegen.

Das Bild lässt sich nun deutlich einfacher nach dem Farbtonwert filtern (hue). Um gute Parameter zu bekommen geht man schrittweise vor. 1) Farbtonwertebereich einschränken, 2) Sättigungsbereich einschränken 3) Helligkeitsbereich einschränken. Man startet mit einem Filterbereich von `inRange(image_baboon_hsv, (0, 0, 0), (255, 255, 255))` und passt die Schwellen von rechts nach links an. Man iteriert sich dann zu: `image_baboon_hsv, (14, 220, 60), (25, 255, 255)`

```
In [8]: # konvertiere RGB zu HSV
image_baboon_hsv = cv2.cvtColor(image_baboon, cv2.COLOR_RGB2HSV)

## Verändern Sie die Filterwerte durch Anpassen der unteren Schranke und der oberen
jaw = cv2.inRange(image_baboon_hsv, (14, 220, 60), (25, 255, 255))

# Anzeige des Bildes
# Achtung: Plotly kann im HSV-Raum abgespeicherte Bilder nicht darstellen. Das Darg
```

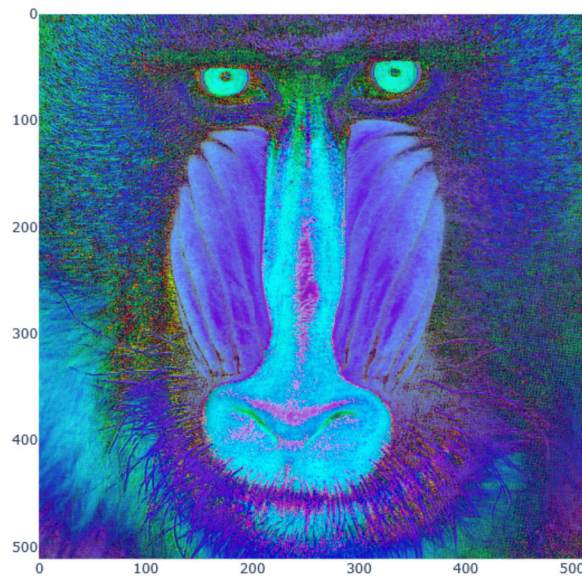
```

height, width = image_baboon_hsv.shape[:2]
fig = px.imshow(image_baboon_hsv, color_continuous_scale='hsv', title='Bild im HSV
fig.update_layout(width=width, height=height + 50, margin=dict(l=0, r=0, t=50, b=0)
fig.show()

# Anzeige der Nase
height, width = jaw.shape[:2]
fig = px.imshow(jaw, color_continuous_scale='gray', title='Bild im HSV Raum')
fig.update_layout(width=width, height=height + 50, margin=dict(l=0, r=0, t=50, b=0)
fig.show()

```

Bild im HSV Raum



Erkennung von Verkehrszeichen

Aufgabe

Nutzen Sie Ihre Erkenntnisse nun für die Erkennung von Verkehrszeichen. Im folgenden Bild sind die Verkehrszeichen:

```
In [9]: image_verkehr=cv2.imread('./Images/verkehr.jpg', cv2.IMREAD_COLOR_RGB)
```

```

# konvertiere RGB zu HSV
image_verkehr_hsv = cv2.cvtColor(image_verkehr, cv2.COLOR_RGB2HSV)

```

```

In [10]: # Anzeige der Bilder
# Achtung: Plotly kann im HSV-Raum abgespeicherte Bilder nicht darstellen. Das Darg
fig = px.imshow(image_verkehr, color_continuous_scale='hsv', title='Bild im RGB Rau
fig.update_layout(width = 500, height=400, margin=dict(l=0, r=0, t=50, b=0))
fig.show()

# Anzeige der Nase
fig = px.imshow(image_verkehr_hsv, color_continuous_scale='gray', title='Bild im HS
fig.update_layout(width = 500, height=400, margin=dict(l=0, r=0, t=50, b=0))
fig.show()

```

```
In [11]: #####
##### ENTER CODE HERE
#####

#filter_vorfahrt =
#filter_zebra =
#filter_tempo =

## Verändern Sie die Filterwerte durch Anpassen der unteren Schranke und der oberen
filter_vorfahrt = cv2.inRange(image_verkehr_hsv, (16, 170, 50), (22, 255, 255))
filter_zebra = cv2.inRange(image_verkehr_hsv, (100, 180, 50), (110, 255, 255))
filter_tempo = cv2.inRange(image_verkehr_hsv, (170, 100, 50), (178, 255, 254))

#
fig = px.imshow(filter_vorfahrt, color_continuous_scale='gray', title='Filter Vohrf
fig.update_layout(width = 500, height=400, margin=dict(l=0, r=0, t=50, b=0))
fig.show()

fig = px.imshow(filter_zebra, color_continuous_scale='gray', title='Filter Zebra')
fig.update_layout(width = 500, height=400, margin=dict(l=0, r=0, t=50, b=0))
fig.show()

fig = px.imshow(filter_tempo, color_continuous_scale='gray', title='Filter Tempo')
fig.update_layout(width = 500, height=400, margin=dict(l=0, r=0, t=50, b=0))
fig.show()
```

Graustufenbilder

Aufgabe

Code Generierung mit Hilfe generativer KI

Wandeln Sie das Bild `image_verkehr` in ein Graustufenbild um. Verwenden Sie dazu keinen vorgefertigten Befehl des Bildverarbeitungspaketes `opencv (cv2.)`. Dokumentieren Sie Ihren Code sodass die Grundidee verstanden werden kann. Vergleichen Sie Graustufenbild mit dem Ergebniss der `OpenCV Funktion cv2.cvtColor(image_verkehr, cv2.COLOR_RGB2Gray)` indem Sie die beiden auf unterschiedliche Weise erzeugten Graustufenbilder voneinander abziehen. Bringen Sie alle 3 Bilder danach zur Anzeige.

```
In [13]: #####
##### ENTER CODE HERE
#####

# img_gray = np.dot(image_verkehr[:, :, :3], [0.33, 0.33, 0.33])

#img_gray = 1/3*(image_verkehr[:, :, 0] + image_verkehr[:, :, 1] + image_verkehr[:, :, 2])

#img_gray = image_verkehr[:, :, 0]*0.33 + image_verkehr[:, :, 1]*0.33 + image_verkehr[:, :, 2]*0.33
```

```

img_gray = image_verkehr[:, :, 0]*0.299 + image_verkehr[:, :, 1]*0.587 + image_verkehr[:, :, 2]*0.114
img_gray = img_gray.astype(np.uint8)

img_gray_cv = cv2.cvtColor(image_verkehr, cv2.COLOR_RGB2GRAY)

difference = img_gray.astype(np.int64) - img_gray_cv.astype(np.int64)

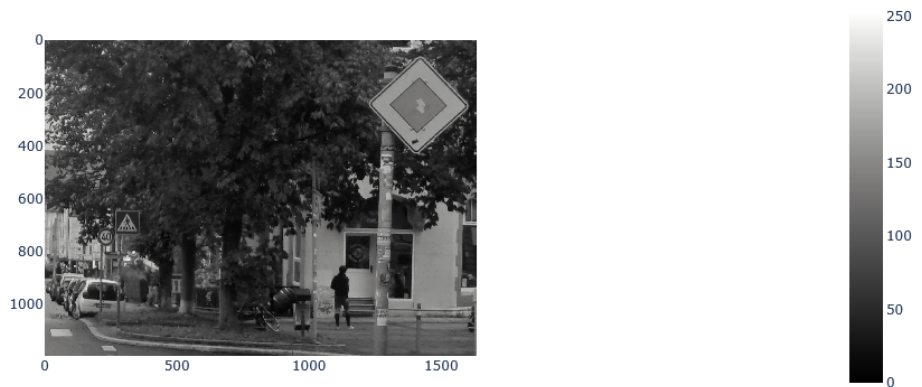
fig = px.imshow(img_gray, color_continuous_scale='gray', title='img_gray')
fig.update_layout(width = 500, height=400, margin=dict(l=0, r=0, t=50, b=0))
fig.show()

fig = px.imshow(img_gray_cv, color_continuous_scale='gray', title='img_gray_cv')
fig.update_layout(width = 500, height=400, margin=dict(l=0, r=0, t=50, b=0))
fig.show()

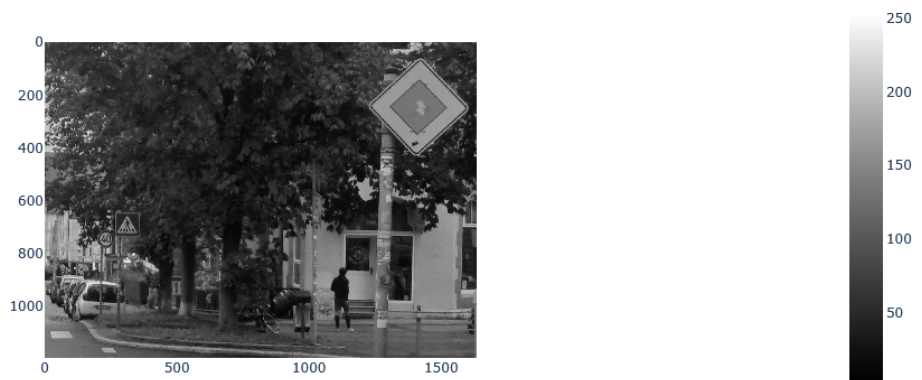
fig = px.imshow(difference, color_continuous_scale='gray', title='difference')
fig.update_layout(width = 500, height=400, margin=dict(l=0, r=0, t=50, b=0))
fig.show()

```

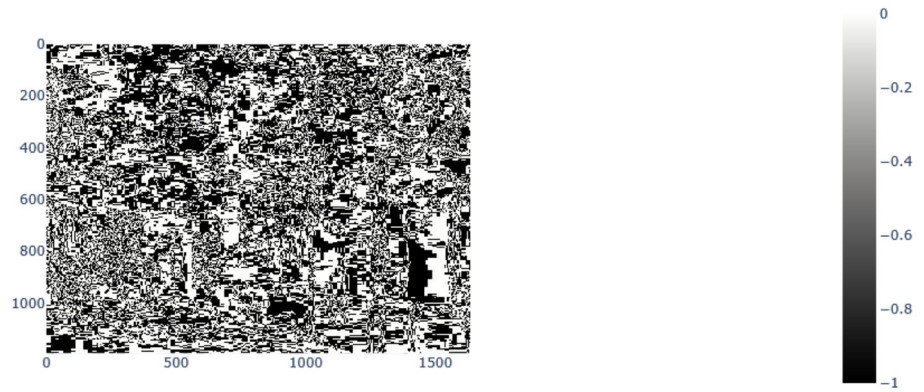
img_gray



img_gray_cv



difference



In []: